

0.NumPy

NumPy(Numerical Python) 是一个开源的Python科学计算库，用于快速处理任意维度的数组。并提供了高效的多维数组（ndarray）对象和各种用于数组计算的函数和工具。在NumPy中，数组是基本的数据结构，可以用于存储和处理任何数值类型的数据。

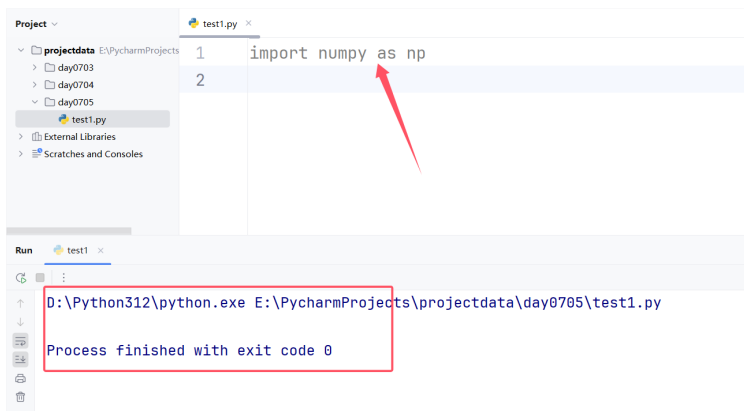
NumPy支持常见的数组和矩阵操作。对于同样的数值计算任务，使用NumPy比直接使用Python要简洁的多。二维数组=矩阵。

官方网站: <https://numpy.org/>

需要下载后才能使用，下载命令如下：

```
1 pip install numpy
2 pip install -i https://pypi.tuna.tsinghua.edu.cn/simple/ xxxx
3 pip install -i https://pypi.tuna.tsinghua.edu.cn/simple/ numpy
```

下载完成后，导入包，进行运行，无报错，则证明下载成功：



1.NumPy数据类型

1	<code>bool_</code>	布尔型 (True、False)
2	<code>int_</code>	默认的整数类型
3	<code>intc</code>	与C的int类型一样，一般是int32或者int64
4	<code>intp</code>	用于索引的整数类型
5	<code>int8</code>	字节 (-128--127)
6	<code>int16</code>	整数 (-32768-32767)
7	<code>int32</code>	整数
8	<code>int64</code>	整数
9	<code>uint8</code>	无符号整数 (0--255)
10	<code>uint16</code>	无符号整数
11	<code>uint32</code>	无符号整数
12	<code>uint64</code>	无符号整数
13	<code>float_</code>	浮点数 (float64)
14	<code>float16</code>	半精度浮点数
15	<code>float32</code>	单精度浮点数
16	<code>float64</code>	双精度浮点数
17	<code>complex_</code>	复数 complex128
18	<code>complex64</code>	
19	<code>complex128</code>	

2.NumPy数组属性

Numpy数组的维数成为：秩。秩就是轴的数量，既是数组的维度，一维数组的秩为1，二维数组的秩为2，以此类推。

Numpy中，每一个线性的数组成为一个轴 (axis)，就是维度。很多时候：

axis=0：表示沿着第0轴进行操作，既对每一列进行操作。

axis=1：表示沿着第1轴进行操作，既对每一行进行操作。

3.NumPy的矩阵

函数reshape()可以将数组转为矩阵。

```
1 import numpy as np
2 a=np.arange(15).reshape(3,5)
3 print(a)
4 a
```

arange(15):表示从0开始产生15个数。

reshape(3,5):表示几行几列。

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14

4.NumPy数组创建

1.一维数组和二维数组的创建

```
1 data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])#一维数组
2 print(data)
3
4 data = np.array([[1, 2, 3], [4, 5, 6]])#二维数组=矩阵
5 print(data)
```

2.创建全0数组

```
1 data=np.zeros(shape=(3, 5), dtype=int)
2 print(data)
```

3.创建全1数组

```
1 data=np.ones(shape=(3, 5), dtype=int)
2 print(data)
```

4.创建全空数组

```
1 data=np.empty(shape=(3, 5), dtype=int)
2 print(data)
```

5.创建有连续序列的数组

```
1 data=np.arange(0,20,2)#0-20之间的数据，步长为2
2 print(data)
```

6.创建有连续间隔的数组

```
1 data=np.linspace(0, 20, 5)#0-20之间的数据，步长为5
2 data
```

7.创建随机数组

```
1 data = np.random.rand(3, 4)#随机产生0-1之间的数据
2 print(data)
```

```
1 data=np.random.randint(1,101,(5,5),dtype=int)
2 print(data)
```

8.改变数组形状

```
1 data1=[1,2,3,4,5]#一维
2 data2=[1,2,3,4,5]
3 data=np.array([data1,data2])#二维
4 print(data)
5 print(data.shape)#(2, 5)
6 #改变形状
7 data=data.reshape(5,2)
8 print(data)
9 print(data.shape)#(5, 2)
```

9.数组转置

```
1 data=np.array([[1,2,3],[4,5,6],[7,8,9]])#二维数组
2 print(data)
3 print(data.T)
```

5.NumPy数组显示操作

```
1 data=np.array([[1,2,3],[4,5,6],[7,8,9]])#二维数组
2 print(data.shape)#维度是多少 (3, 3)
3 print(data.ndim)#有几个维度 2
4 print(data.dtype.name)#查看数组的类型 int32
5 print(data.size)#查看数据组有多少个元素 9
```

6.NumPy基本计算

1.一维计算

```

1  a=np.array([20,30,40,50])
2  b=np.array([1,2,3,4])#0,1,2,3
3  #减法
4  c=a-b
5  print(c)#[19 28 37 46]
6  #加法
7  d=a+b
8  print(d)#[21 32 43 54]
9  #乘法
10 e=a*b
11 print(e)#[ 20  60 120 200]
12 #除法
13 f1=a/b
14 f2=a//b
15 print(f1)#[20.          15.          13.33333333 12.5          ]
16 print(f2)#[20 15 13 12]
17 #平方
18 print(b**2)#[ 1  4  9 16]
19 #判断
20 print(a<35)#[ True  True False False]

```

2.二维计算

```

1  A = np.array([[1, 1], [3, 1]])
2  B = np.array([[2, 1], [3, 4]])
3  print(A)
4  print(B)
5  print('-----加减乘除-----')
6  print(A + B)
7  print(A - B)
8  print(A * B)
9  print(A // B)
10 print('-----')
11 print(A.dot(B))#表示做矩阵的乘法运算
12 print(A<2)
13 print(b**4)

```

3.其他计算

```
1 B=np.array([1,2,3])# 1,2,3
2 print(np.exp(B))
3 print(np.log(B))
4 print(np.log1p(B))
5 print(np.log10(B))
6 print(np.power(B, 2))
7 print(np.sqrt(B))
```

```
1 C=np.arange(12).reshape(3,4)
2 print(C)
3 print('-----')
4 print(C.sum(axis=0))#每列（垂直）
5 print(C.sum(axis=1))#每行（水平）
6 print(C.min(axis=0))
7 print(C.min(axis=1))
8 print(C.max(axis=0))
9 print(C.max(axis=1))
```

7.NumPy数组连接

1.concatenate()函数连接两个或者多个数组

```
1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3 result=np.concatenate((a, b), axis=0)
4 print(result.shape)
5 result=np.concatenate((a, b), axis=1)
6 print(result.shape)
```

2.statck()按照指定轴的方向堆叠

```

1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3 result=np.stack((a, b), axis=0)
4 print(result)
5 print(result.shape)#(2, 2, 2)
6 print(result.ndim)#3
7 result=np.stack((a, b), axis=1)
8 print(result)
9 print(result.shape)#(2, 2, 2)
10 print(result.ndim)#3

```

3.hstack(): 水平堆叠、vstack(): 垂直堆叠

```

1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3 result=np.hstack((a, b))
4 print(result)
5 result=np.vstack((a, b))
6 print(result)

```

8.NumPy数组拆分

1.split()函数沿着特定的轴拆分数组

```

1 a=np.arange(12)
2 print(a)
3 b=np.split(a,6)#拆分的份数需要跟数组的长度有倍数关系
4 print(b)

```

2.hsplit()函数水平拆分、vsplit()函数垂直拆分

```

1 a=np.arange(8)
2 result=np.hsplit(a,2)
3 print(result)
4
5 b=np.array([[1,2],[3,4]])
6 result=np.vsplit(b,2)
7 print(result)

```

9.NumPy数组切片

一维操作

```
1 a = np.arange(10)
2 print(a)
3 print(a[2])
4 print(a[2:5])
5 print(a[:-1])
6 print(a[-4])
```

二维操作

```
1 a=np.array([[1, 2,3], [4,5,6], [7,8,9]])
2 print(a)
3 print(a.shape)
4 #取第1行后面的所有行、所有列
5 print(a[1:])
6 #取第1行第1列的数据
7 print(a[1,1])
8 #取第0行到第2行(不包括第2行), 第一列的数据
9 print(a[0:2,1])#[2 5]
10 #取所有行, 第1列数据
11 print(a[:,1])#[2 5 8]
```

10.NumPy数组统计函数

amin():计算轴方向的最小值

amax():计算轴方向的最大值

```
1 a = np.array([[3, 7, 5], [4, 6, 8], [2, 4, 9]])
2 print(a)
3 r1=np.amin(a,axis=0)
4 r2=np.amin(a,axis=1)
5 print(r1)
6 print(r2)
7
8 r11=np.amax(a,axis=0)
9 r22=np.amax(a,axis=1)
10 print(r11)
11 print(r22)
```

median():中位数

mean():平均数

```
1 a = np.array([[3, 7, 5], [4, 6, 8], [2, 4, 9]])
2 r1=np.median(a)#中位数
3 print(r1)
4
5 r12=np.median(a,axis=0)#中位数
6 print(r12)
7
8 r13=np.median(a,axis=1)#中位数
9 print(r13)
10
11
12 r21=np.mean(a)#平均数
13 print(r21)
14
15 r22=np.mean(a,axis=0)#平均数
16 print(r22)
17
18 r33=np.mean(a,axis=1)#平均数
19 print(r33)
```

average():加权平均

```
1 a = np.array([1, 2, 3, 4])#数组-数值
2 w = np.array([4, 3, 2, 1])#数组-权重
3 #加权平均值:  $1*4+2*3+3*2+4*1=4+6+6+4=20/10=2$ 
4
5 r1=np.average(a,weights=w)
6 print(r1)#2
```

var(): 计算方差。每个数与平均数的差, 再平方和

std(): 标准差。方差的开方。

```

1 a = np.array([1, 2, 3, 4])
2 #计算方差
3 # 均值=2.5
4 # (1-2.5)^2+(2-2.5)^2+(3-2.5)^2+(4-2.5)^2=5/4=1.25
5 print(np.var(a))#1.25
6 #标准差
7 print(np.std(a))#1.118

```

11.练习

1.对一个5*5的随机矩阵做归一化。

$$Z = (X - \min) / (\max - \min)。$$

```

1 Z = np.random.randint(1, 1000, (5, 5), dtype=int)
2 print(Z)
3 zMax = Z.max()
4 zMin = Z.min()
5 print(zMin, zMax)
6 #Z= (X-min) / (max-min)
7 Z = (Z - zMin) / (zMax - zMin)
8 print(Z)

```

2.创建一个10*10的随机数组，要求找到他的最大值和最小值。

```

1 Z = np.random.randint(1, 1000, (10, 10), dtype=int)
2 zMax = Z.max()
3 zMin = Z.min()
4 print(zMin, zMax)

```

12.作业

写完可以一起将结果截图发到QQ群。

5. 水仙花数指一个数等于它各个数字的立方和，例如， $153 = 1^3 + 5^3 + 3^3$ 。要求通过编写程序，输出所有 3 位数的水仙花数。

6. 经典的百鸡百钱问题：1 只公鸡 5 元，1 只母鸡 3 元，3 只小鸡 1 元。现在花 100 元买了 100 只鸡，编写程序，求公鸡、母鸡、小鸡各买了多少只？

7. 判断 2~100 的数是否为素数，要求输出类似下面的结果。

```

3 是素数
5 是素数
7 是素数
11 是素数

```

第5题

```
1 def test4():
2     for num in range(100, 1000):
3         i = num // 100 # 百位数
4         j = (num - i * 100) // 10 # 十位数
5         k = (num - i * 100 - j * 10) # 个位数
6         if num == pow(i, 3) + pow(j, 3) + pow(k, 3):
7             print(num)
8 test4()
```

第6题

```
1 def test5():
2     for cock in range(0, 21): # 公鸡
3         for hen in range(0, 34): # 母鸡
4             chick = 100 - cock - hen
5             if cock * 5 + hen * 3 + chick / 3 == 100:
6                 print("公鸡:" , cock, "母鸡:" , hen, "小鸡:" , chick)
7 test5()
```

第7题

```
1 def test6():
2     i = 2
3     j = 2
4     for j in range(2, 100):
5         for i in range(2, j):
6             if j % i == 0:
7                 break
8             elif (j - 1) == i:
9                 print("%d是素数" % j)
10 test6()
```

